

CSE 493S Empirical Methods Assignment

Progress Report: Finished Deliverables

Aadyant Maity and Rahul Bonthu

May 1, 2026

Part 0: implementation

Part 0 is basically the codebase the rest of the assignment builds on. Everything lives under `deliverables/part_0/`: the usual scripts (`train.py`, `model.py`, `inference.py`, `tokenizer_utils.py`, `arith_data.py`) plus `part_0_1_contract.py` for the autograder-facing API. We sanity-check with the quick configs under `deliverables/part_0/configs/`, for example:

```
python train.py --config configs/sanity_quick.yaml
python train.py --config configs/sanity_suffix_quick.yaml
```

(run from the `EmpiricalHW/` root, or point paths appropriately). Nothing fancy to report here besides “it trains and the contract loads checkpoints”—the interesting experiments start in Part 1.

Part 1.1: modular arithmetic datasets

For Part 1.1 we generated text files of equations $a \text{ op } b = c$ with arithmetic mod p . Addition and subtraction iterate all pairs (a, b) with $0 \leq a, b < p$. Division skips $b = 0$ and uses $b \in \{1, \dots, p-1\}$, so division sets are a bit smaller than add/sub for the same p . We shuffled once with seed 0, then split each file **80% / 10% / 10%** train, validation, and test in that order. Every `meta.json` matches the lines in the matching `.txt` files.

Split sizes

These counts match what is on disk under `deliverables/part_1/1.1/data/`.

Table 1: Number of lines per split for each operator and modulus.

p	op	train	val	test
97	+ / -	7527	940	942
97	/	7449	931	932
113	+ / -	10215	1276	1278
113	/	10124	1265	1267

To regenerate from the course workflow:

```
python arith_data.py --seed 0
```

Part 1.2: addition mod 97 (one-layer, deliverable run)

Part 1.2 is the addition-mod-97 setup with a one-layer model. Our submitted bundle lives under `deliverables/part_1/1.2/checkpoints/p97_add_1L_restart1_seed101/`, with final weights `ckpt_100000.pt` (`max_steps = 100000`, `save_every = 10000`), plus `tokenizer.json`, `metrics.csv`, and resolved config metadata.

Training curves (seed 101)

Figure 1 shows train/val/test loss, token accuracy, and equation accuracy for steps 0–100000 only (we pass `--max-step 100000` to `plot_metrics.py` so the figure matches the deliverable budget exactly).

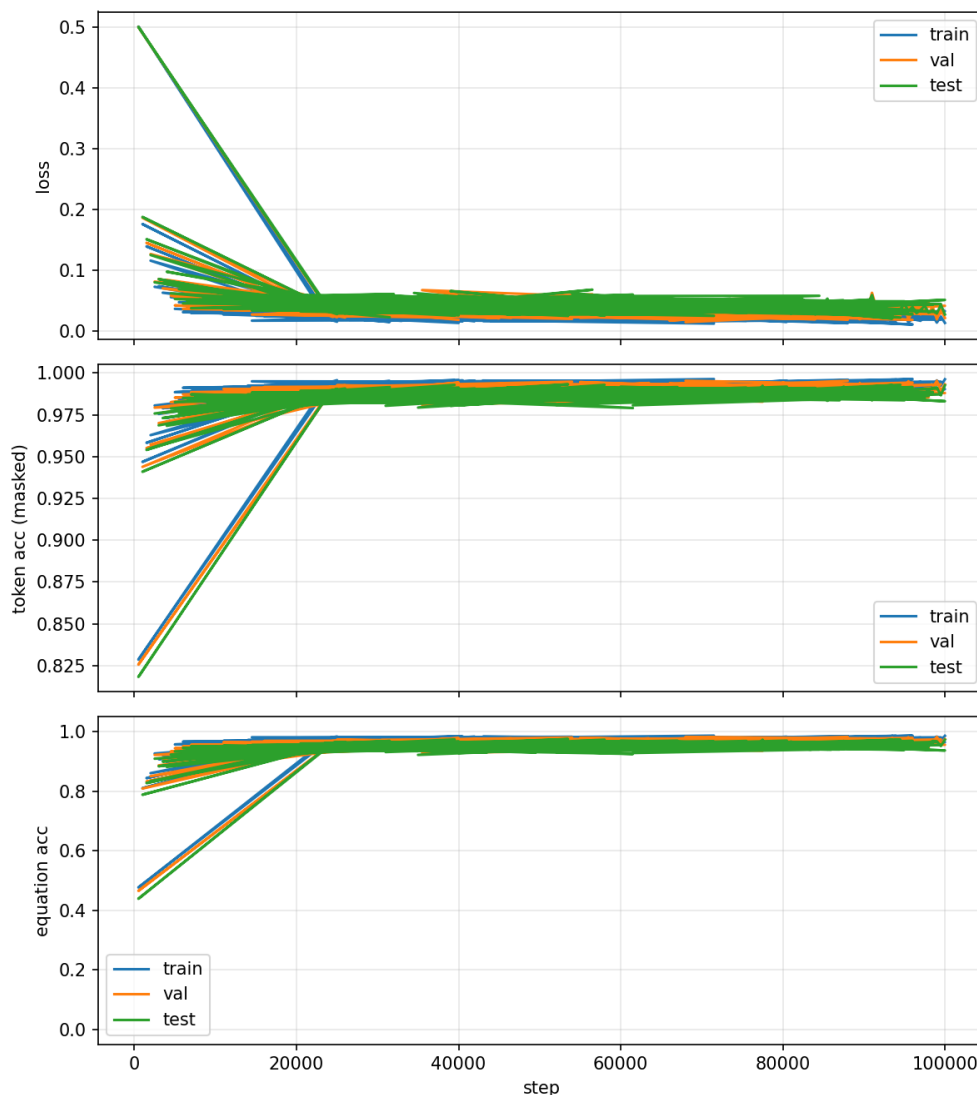


Figure 1: Part 1.2 deliverable run: $p = 97$, addition, one layer, seed 101, 100k steps.

Numbers at the last logged step (seed 101)

At step 100000 (last row of the deliverable `metrics.csv`), we see roughly:

- Train loss ≈ 0.0136 , train token acc ≈ 0.996 .
- Val loss ≈ 0.0218 , val token acc ≈ 0.993 .
- Test loss ≈ 0.0275 , test token acc ≈ 0.993 .
- Train / val / test equation acc ≈ 0.985 , 0.974 , and 0.972 .

Part 1.3: division mod 97 (grokking-style run)

Part 1.3 trains on division modulo 97. The deliverable bundle is under `deliverables/part_1/1.3/checkpoints/part1_3_div_p97/`, with final weights `ckpt_300000.pt`, `tokenizer.json`, `metrics.csv`, and resolved config metadata.

Training curves

Figure 2 shows loss and accuracy vs. step from `metrics.csv`. We generated the PNG with `plot.metrics.py` from that run's CSV (repo-root script, `--max-step` optional).

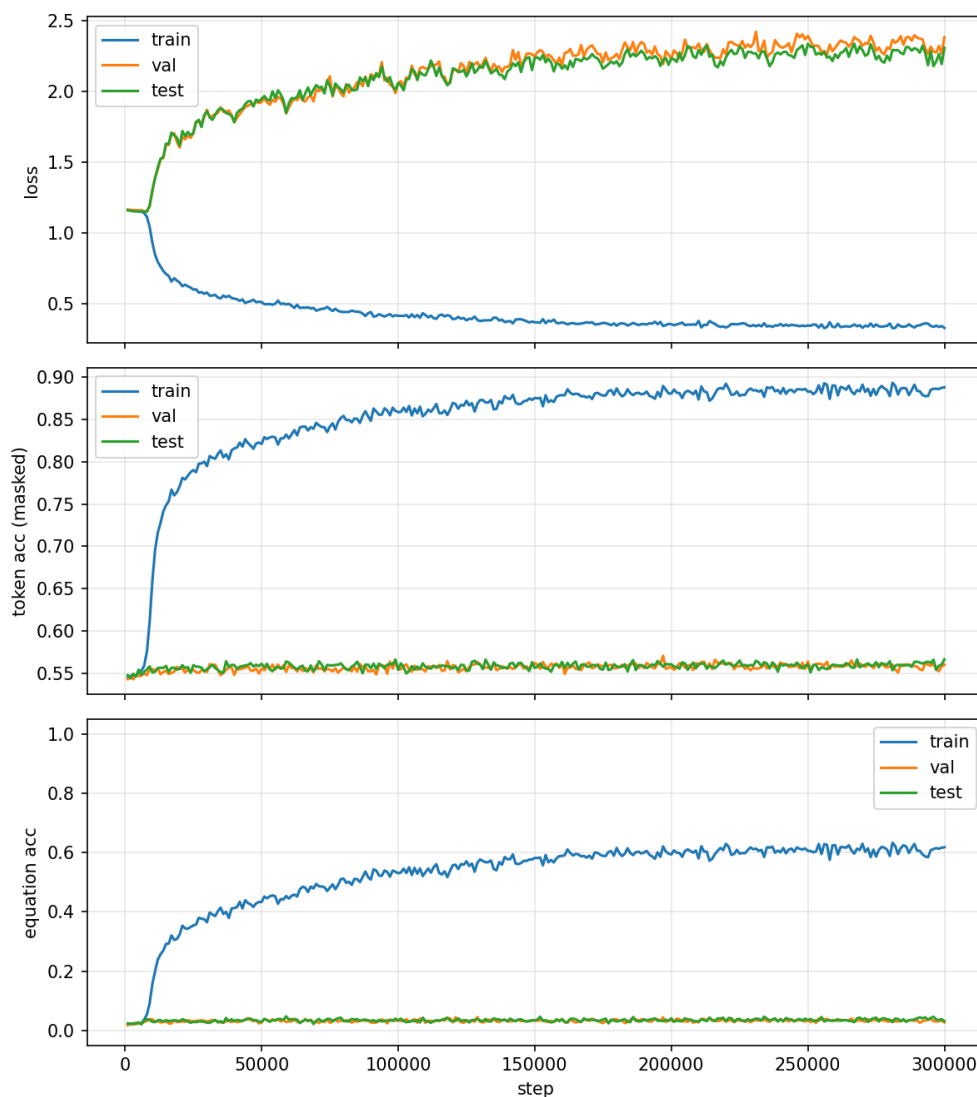


Figure 2: Training / validation / test curves for the division mod 97 experiment (`part1_3_div_p97`).

Numbers at the last logged step

At step 300000 (last row of `metrics.csv`), the CSV reports roughly:

- Train loss ≈ 0.331 , train token acc ≈ 0.888 .
- Val loss ≈ 2.38 , val token acc ≈ 0.560 .
- Test loss ≈ 2.31 , test token acc ≈ 0.566 .
- Train equation acc ≈ 0.618 ; val/test equation acc stayed low on the last rows.

We are not over-interpreting these here—the figure is the main takeaway for us until we write up discussion elsewhere.

How to run inference

From `deliverables/part_1/1.3/code/`, we can use either free-form generation or `predict_answer` on integers. Example:

```
python inference.py --checkpoint_dir ../checkpoints/part1_3_div_p97 \
    --prompt "3 / 5 = "
```

Part 1.4: ablations (division mod 97)

Part 1.4 compares two tweaks to the Part 1.3-style division setup ($p = 97$, one layer, 300k steps, seed 42). Each ablation has its own checkpoint folder under `deliverables/part_1/1.4/checkpoints/`.

Dropout (dropout = 0.2, weight_decay = 1.0)

Bundle: `part_1/1.4/checkpoints/part1_4_ablation_dropout/`.

Figure 3: `plot.metrics.py` with `--max-step 300000`.



Figure 3: Part 1.4 dropout ablation: division mod 97, dropout = 0.2, 300k steps.

At step 300000 (last row of that run's `metrics.csv`):

- Train loss ≈ 1.16 , train token acc ≈ 0.546 .
- Val loss ≈ 1.16 , val token acc ≈ 0.540 .

- Test loss ≈ 1.16 , test token acc ≈ 0.548 .
- Train / val / test equation acc ≈ 0.021 , 0.016 , and 0.021 .

Low weight decay (`weight_decay = 0.1`, `dropout = 0.0`)

Bundle: `part_1/1.4/checkpoints/part1_4_ablation_wd_low/`.

Figure 4: same `plot_metrics.py` setup with that run's `metrics.csv`.

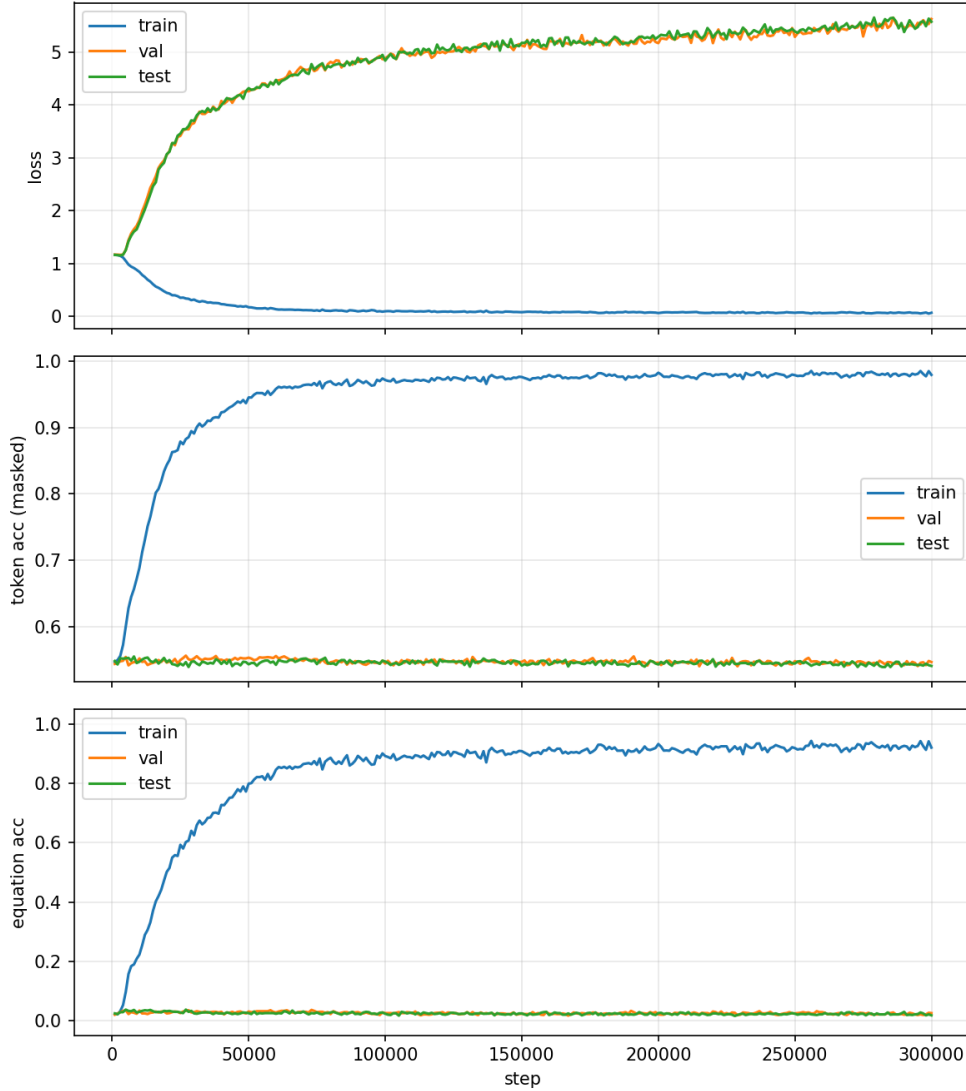


Figure 4: Part 1.4 low weight decay ablation: division mod 97, `weight_decay = 0.1`, 300k steps.

At step 300000 (last row of that run's `metrics.csv`):

- Train loss ≈ 0.065 , train token acc ≈ 0.979 .
- Val loss ≈ 5.63 , val token acc ≈ 0.546 .
- Test loss ≈ 5.58 , test token acc ≈ 0.541 .
- Train equation acc ≈ 0.921 ; val / test equation acc ≈ 0.025 and 0.017 .

Part 2.1: Test-Time Scaling Warm-Up

For the warm-up in Part 2.1, we evaluated the Qwen3-4B model on the AIME 2024 dataset (30 problems) using greedy decoding. We measured the zero-shot accuracy under two conditions: allowing the model to use its thinking trace and suppressing the thinking trace (forcing an immediate answer).

Greedy Accuracy Baseline

As shown in Table 2, the model achieved 0% accuracy across both exact match and flexible extraction modes for all 30 problems.

Table 2: Greedy decoding accuracy on AIME 2024 (n=30).

Condition	Exact Match Accuracy	Flexible Extract Accuracy
With Thinking	0.000	0.000
Without Thinking	0.000	0.000

The uniform failure at zero accuracy is a direct result of the model’s behavior during greedy decoding (*do_sample = False*). Specifically, we observed a phenomenon of “mode collapse” or token attraction: once the thinking trace concluded, the model frequently fixated on a single repetitive integer (e.g., “78” or “108”) across many different prompts. Without the variance provided by sampling, the model was unable to escape these local minima to arrive at the correct gold answers for the highly complex AIME problems.

Thinking Length Distribution

While accuracy was poor, the model successfully generated varied reasoning traces when thinking was enabled. We parsed these traces to measure the distribution of thinking tokens. Figure 5 illustrates the spread of reasoning compute used by the model before emitting a final answer.

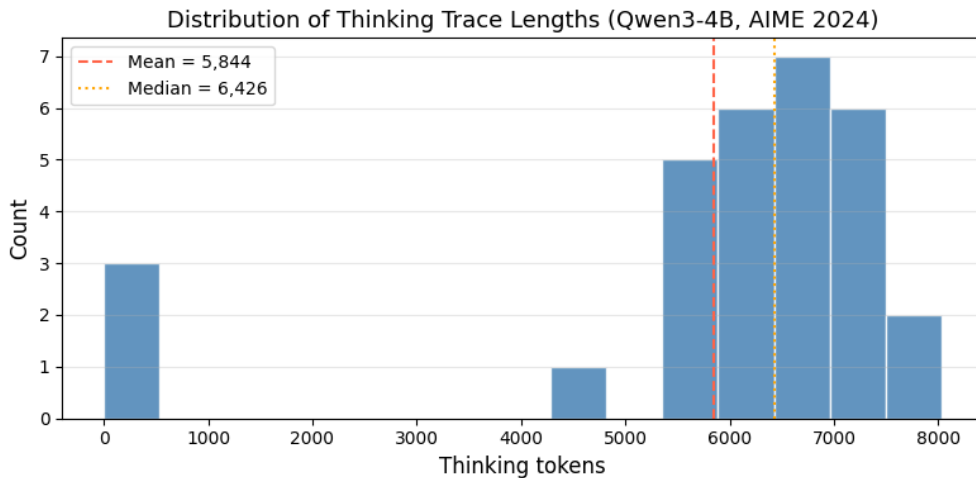


Figure 5: Distribution of thinking trace lengths for Qwen3-4B on AIME 2024 problems.

The distribution characteristics are summarized below:

- **Mean:** $\approx 5,844$ tokens
- **Median:** $\approx 6,333$ tokens
- **Minimum:** 0 tokens (for problems where the model bypassed thinking)
- **Maximum:** 8,032 tokens

The varied lengths suggest that the model’s internal reasoning process scales dynamically with problem complexity, even when the final output fails to reach the correct answer. This baseline serves as the starting point for our subsequent scaling experiments in Part 2.2.